Homework: Numerical Integration

Problem 1: Implementing and Comparing Quadrature Rules

(a) Implement the composite trapezoidal rule and Simpson's rule with the following signatures:

```
import numpy as np

def trapezoidal(f, a, b, n):
    """Composite trapezoidal rule with n subintervals.

    Parameters
    ----------
    f : callable
        Function to integrate.
    a, b : float
        Integration limits.
    n : int
        Number of subintervals.

    Returns
    -------
    float
        Approximate integral value.
    """
    x = np.linspace(a, b, n + 1)
    y = f(x)
    h = (b - a) / n
    return float((h / 2) * (2 * np.sum(y) - y.item(0) - y.item(-1)))

def simpsons(f, a, b, n):
    """Composite Simpson's rule with n subintervals (n must be even).

    Parameters
    ----------
    f : callable
        Function to integrate.
    a, b : float
        Integration limits.
    n : int
        Number of subintervals (must be even).

    Returns
    -------
    float
        Approximate integral value.
    """
    if n % 2 != 0:
        raise ValueError("n must be even for Simpson's rule.")

    x = np.linspace(a, b, n + 1)
    y = f(x)
    h = (b - a) / n
    y_first = y.item(0)
    y_last = y.item(-1)
    sum_odd = sum(y.item(i) for i in range(1, n, 2))
    sum_even = sum(y.item(i) for i in range(2, n, 2))
    s = y_first + y_last + 4 * sum_odd + 2 * sum_even
    return float((h / 3) * s)

# --- Testing and Error Reporting ---

f = lambda x: np.exp(-x**2)
a, b = 0, 1
true_val = 0.746824132812427
ns = (4, 16, 64, 256)
print(f"{'n':>4} | {'Trap. Error':>15} | {'Simp. Error':>15}")
print("-" * 40)

for n in ns:
    val_t = trapezoidal(f, a, b, n)
    val_s = simpsons(f, a, b, n)
```

```
    err_t = float(abs(val_t - true_val))
    err_s = float(abs(val_s - true_val))

    print(f"{n:4d} | {err_t:15.8e} | {err_s:15.8e}")
```

```
   n |     Trap. Error |     Simp. Error
-------------------------------------------
   4 |  3.84003501e-03 |  3.12469786e-05
  16 |  2.39536024e-04 |  1.24623303e-07
  64 |  1.49691746e-05 |  4.87245466e-10
 256 |  9.35566275e-07 |  1.90336635e-12
```

Test both on $I = \int_0^1 e^{-x^2}\,dx$ with $n = 4, 16, 64, 256$. For each n, report the absolute error.

(b) For each method, verify the theoretical convergence rate by computing the ratio of consecutive errors as n doubles. The trapezoidal rule should show a ratio near 4 (since error is $O(n^{-2})$) and Simpson's rule should show a ratio near 16 (since error is $O(n^{-4})$).

```
ns = (4, 8, 16, 32, 64, 128, 256)

print(f"{'n':>4} | {'Trap. Error':>15} | {'Trap. Ratio':>12} || {'Simp. Error':>15} | {'Simp. Ratio':>12}")
print("-" * 75)

prev_err_t = None
prev_err_s = None

for n in ns:
    val_t = trapezoidal(f, a, b, n)
    val_s = simpsons(f, a, b, n)

    err_t = float(abs(val_t - true_val))
    err_s = float(abs(val_s - true_val))

    if prev_err_t is not None:
        ratio_t = float(prev_err_t / err_t)
        ratio_s = float(prev_err_s / err_s)
        print(f"{n:4d} | {err_t:15.8e} | {ratio_t:12.4f} || {err_s:15.8e} | {ratio_s:12.4f}")
    else:
        print(f"{n:4d} | {err_t:15.8e} | {'-':>12} || {err_s:15.8e} | {'-':>12}")

    prev_err_t = err_t
    prev_err_s = err_s
```

```
   n |     Trap. Error |  Trap. Ratio ||     Simp. Error |  Simp. Ratio
---------------------------------------------------------------------------
   4 |  3.84003501e-03 |            - ||  3.12469786e-05 |            -
   8 |  9.58517967e-04 |       4.0062 ||  1.98771504e-06 |      15.7200
  16 |  2.39536024e-04 |       4.0016 ||  1.24623303e-07 |      15.9498
  32 |  5.98781601e-05 |       4.0004 ||  7.79455811e-09 |      15.9885
  64 |  1.49691746e-05 |       4.0001 ||  4.87245466e-10 |      15.9972
 128 |  3.74227081e-06 |       4.0000 ||  3.04540837e-11 |      15.9993
 256 |  9.35566275e-07 |       4.0000 ||  1.90336635e-12 |      16.0001
```

Problem 2: Variance Reduction with Antithetic Variables

Standard Monte Carlo integration draws independent uniform samples to estimate an integral. Antithetic variables is a variance reduction technique that pairs each sample U_i with its "mirror" $a + b - U_i$ on the interval $[a, b]$. When the integrand is monotone, these pairs are negatively correlated, which reduces the variance of the estimator.

(a) Implement a Monte Carlo integrator that uses antithetic variables. For each of n uniform draws U_i on $[a, b]$, compute both $f(U_i)$ and $f(a + b - U_i)$, then average the pair. The estimate is the mean of these n pairwise averages (using $2n$ total function evaluations).

```
import numpy as np

def mc_antithetic(f, a, b, n, seed=None):
    """Monte Carlo integration with antithetic variables on [a, b].

    Parameters
    ----------
    f : callable
        Function to integrate.
    a, b : float
        Integration limits.
```

```
    n : int
        Number of antithetic pairs (total function evaluations = 2n).
    seed : int or None
        Random seed for reproducibility.

    Returns
    -------
    estimate : float
        Antithetic MC estimate of the integral.
    se : float
        Standard error of the estimate.
    """
    # Your code here
    pass
```

Test on $I = \int_0^1 e^{-x^2}\,dx$ with $n = 500, 5000, 50000$ pairs (use `seed=42` with `numpy.random.default_rng`). For each n, report the estimate, standard error, and absolute error.

```
import numpy as np

def mc_antithetic(f, a, b, n, seed=None):
    """Monte Carlo integration with antithetic variables on [a, b].

    Parameters
    ----------
    f : callable
        Function to integrate.
    a, b : float
        Integration limits.
    n : int
        Number of antithetic pairs (total function evaluations = 2n).
    seed : int or None
        Random seed for reproducibility.

    Returns
    -------
    estimate : float
        Antithetic MC estimate of the integral.
    se : float
        Standard error of the estimate.
    """
    rng = np.random.default_rng(seed)

    U = rng.uniform(a, b, n)
    U_prime = a + b - U
    Z = (b - a) * (f(U) + f(U_prime)) / 2.0
    estimate = np.mean(Z)
    se = np.std(Z, ddof=1) / np.sqrt(n)
    return float(estimate), float(se)

# --- Testing and Error Reporting ---

f = lambda x: np.exp(-x**2)
a, b = 0.0, 1.0
true_val = 0.746824132812427

ns = (500, 5000, 50000)

print(f"{'n pairs':>8} | {'Estimate':>15} | {'Std Error':>15} | {'Abs Error':>15}")
print("-" * 60)

for n in ns:
    est, se = mc_antithetic(f, a, b, n, seed=42)

    abs_err = float(abs(est - true_val))

    print(f"{n:8d} | {est:15.8f} | {se:15.8e} | {abs_err:15.8e}")
```

```
 n pairs |        Estimate |       Std Error |       Abs Error
------------------------------------------------------------
     500 |      0.74728148 |  1.22446948e-03 |  4.57343040e-04
    5000 |      0.74685366 |  3.99948891e-04 |  2.95298842e-05
   50000 |      0.74683616 |  1.26892788e-04 |  1.20262466e-05
```

(b) For a fair comparison, standard MC with the same computational budget uses $2n$ independent samples. For each value of n above, also compute the standard MC estimate and SE using $2n$ samples (same seed). Report the variance reduction factor $\mathrm{SE}^2_{\text{standard}}/\mathrm{SE}^2_{\text{antithetic}}$ for each n.

```
def mc_standard(f, a, b, num_samples, seed=None):
    """Standard Monte Carlo integration on [a, b]."""
    rng = np.random.default_rng(seed)
    U = rng.uniform(a, b, num_samples)

    Y = (b - a) * f(U)
    estimate = np.mean(Y)
    se = np.std(Y, ddof=1) / np.sqrt(num_samples)

    return float(estimate), float(se)

     # --- Testing and Comparison ---

    f = lambda x: np.exp(-x**2)
    a, b = 0.0, 1.0
    true_val = 0.746824132812427
    ns = (500, 5000, 50000)

print(f"{'n pairs':>8} | {'Anti SE':>14} | {'Std SE (2n)':>14} | {'VRF':>10}")
print("-" * 55)

for n in ns:
    budget = n * 2
    est_anti, se_anti = mc_antithetic(f, a, b, n, seed=42)
    est_std, se_std = mc_standard(f, a, b, budget, seed=42)
    vrf = float((se_std / se_anti)**2)

    print(f"{n:8d} | {se_anti:14.8e} | {se_std:14.8e} | {vrf:10.4f}")
```

```
 n pairs |        Anti SE |    Std SE (2n) |        VRF
-------------------------------------------------------
     500 | 1.22446948e-03 | 6.40156955e-03 |    27.3324
    5000 | 3.99948891e-04 | 2.00504172e-03 |    25.1326
   50000 | 1.26892788e-04 | 6.35161792e-04 |    25.0550
```

(c) Explain why antithetic variables reduce variance for the integrand e^{-x^2} on $[0, 1]$. Describe a type of function for which antithetic variables would provide little or no variance reduction.

Why antithetic variables work for e^{-x^2}:

1. Monotonicity: The integrand $f(x) = e^{-x^2}$ is strictly monotonically decreasing over the interval.
2. Negative Covariance: By construction, the inputs U and $1 - U$ move in exact opposite directions. Because $f(x)$ is monotonic, a large $f(U)$ is deterministically paired with a small $f(1 - U)$. This guarantees a negative covariance: $\mathrm{Cov}(f(U), f(1 - U)) < 0$.
3. Variance Formula: The variance of the antithetic pair is $\frac{1}{2}\mathrm{Var}(f(U)) + \frac{1}{2}\mathrm{Cov}(f(U), f(1 - U))$. The strictly negative covariance term geometrically reduces the total variance compared to standard Monte Carlo with independent samples.

Symmetric Functions: Functions that are perfectly symmetric about the interval's midpoint, such as $f(x) = (x - 0.5)^2$, fail to reduce variance. If $f(x)$ is symmetric, then $f(U) = f(1 - U)$ for all U. This forces perfect positive correlation, where $\mathrm{Cov}(f(U), f(1 - U)) = \mathrm{Var}(f(U))$. The variance of the paired average becomes exactly $\mathrm{Var}(f(U))$. You expend two computational function evaluations but only gain the statistical precision of a single independent sample, effectively halving the algorithmic efficiency.

Problem 3: Monte Carlo Integration

(a) Write a function that estimates $\int_a^b f(x)dx$ by Monte Carlo integration and returns both the estimate and its standard error:

```
def mc_integrate(f, a, b, n, seed=None):
    """Monte Carlo integration on [a, b].

    Parameters
    ----------
    f : callable
        Function to integrate.
    a, b : float
        Integration limits.
    n : int
```

```
        Number of random samples.
    seed : int or None
        Random seed for reproducibility.

    Returns
    -------
    estimate : float
        MC estimate of the integral.
    se : float
        Standard error of the estimate.
    """
    # Your code here
    pass
```

Test on $\int_0^1 e^{-x^2}\,dx$ with $n = 100, 1000, 10000, 100000$ (use `seed=42`). For each n, report the estimate, standard error, and absolute error.

```
import numpy as np

def mc_integrate(f, a, b, n, seed=None):
    """Monte Carlo integration on [a, b].

    Parameters
    ----------
    f : callable
        Function to integrate.
    a, b : float
        Integration limits.
    n : int
        Number of random samples.
    seed : int or None
        Random seed for reproducibility.

    Returns
    -------
    estimate : float
        MC estimate of the integral.
    se : float
        Standard error of the estimate.
    """
    rng = np.random.default_rng(seed)
    U = rng.uniform(a, b, n)
    Y = (b - a) * f(U)
    estimate = np.mean(Y)
    se = np.std(Y, ddof=1) / np.sqrt(n)
    return float(estimate), float(se)

# --- Testing and Error Reporting ---

f = lambda x: np.exp(-x**2)
a, b = 0.0, 1.0
true_val = 0.746824132812427

ns = (100, 1000, 10000, 100000)

print(f"{'n samples':>10} | {'Estimate':>15} | {'Std Error':>15} | {'Abs Error':>15}")
print("-" * 63)

for n in ns:
    est, se = mc_integrate(f, a, b, n, seed=42)
    abs_err = float(abs(est - true_val))

    print(f"{n:10d} | {est:15.8f} | {se:15.8e} | {abs_err:15.8e}")
```

```
 n samples |        Estimate |       Std Error |       Abs Error
---------------------------------------------------------------
       100 |      0.75835314 |  1.88832902e-02 |  1.15290032e-02
      1000 |      0.74799874 |  6.40156955e-03 |  1.17460788e-03
     10000 |      0.74892947 |  2.00504172e-03 |  2.10533349e-03
    100000 |      0.74639593 |  6.35161792e-04 |  4.28199501e-04
```

(b) Verify that the MC error rate is $O(n^{-1/2})$: compute the ratio of standard errors as n increases by a factor of 100 (from 100 to 10000). The SE should decrease by a factor of approximately 10.

```
f = lambda x: np.exp(-x**2)
a, b = 0.0, 1.0

_, se_100 = mc_integrate(f, a, b, 100, seed=42)
_, se_10000 = mc_integrate(f, a, b, 10000, seed=42)

ratio = float(se_100 / se_10000)

print(f"{'SE at n=100':>15}: {se_100:15.8e}")
print(f"{'SE at n=10000':>15}: {se_10000:15.8e}")
print("-" * 33)
print(f"{'Observed Ratio':>15}: {ratio:15.4f}")
print(f"{'Expected Ratio':>15}: {10.0:15.4f}")
```

```
    SE at n=100:  1.88832902e-02
  SE at n=10000:  2.00504172e-03
---------------------------------
 Observed Ratio:          9.4179
 Expected Ratio:         10.0000
```

(c) The $O(n^{-1/2})$ rate appears slow compared to deterministic quadrature. With 256 subintervals, Simpson's rule achieves error around 10^{-12} for this 1D integral, while MC with 100,000 points still has error around 10^{-4}. Why then is MC preferred for high-dimensional integrals?

1. The Degradation of Deterministic QuadratureTo evaluate a d-dimensional integral using a deterministic method like Simpson's rule, we must construct a grid. If we use m points along each of the d axes, the total number of function evaluations is $N = m^d$. Because the 1D error for Simpson's is $O(m^{-4})$, we can substitute $m = N^{1/d}$ to find the error in terms of total computational budget: $O(N^{-4/d})$. In 1D ($d = 1$): Error is $O(N^{-4})$ (Extremely fast). In 10D ($d = 10$): Error is $O(N^{-0.4})$ (Slower than Monte Carlo).
2. The Invariance of Monte CarloThe standard error of Monte Carlo integration is derived from the variance of the samples: $\mathrm{SE} = \frac{\sigma}{\sqrt{N}}$. This rate of $O(N^{-1/2})$ holds true whether we are integrating over a 1-dimensional line or a 1,000-dimensional hypervolume.

Problem 4: Importance Sampling

The following code estimates $P(Z > 4)$ where $Z \sim N(0, 1)$ using naive Monte Carlo and importance sampling with a shifted exponential proposal. Read the code and answer the questions.

```
import numpy as np
from scipy import stats

true_prob = 1 - stats.norm.cdf(4)  # ≈ 3.17e-5

# Naive MC
np.random.seed(42)
z_samples = np.random.normal(0, 1, 100000)
naive_est = np.mean(z_samples > 4)

# Importance sampling with Exp(1) shifted to start at 4
np.random.seed(42)
x_is = np.random.exponential(1.0, 100000) + 4.0
log_weights = stats.norm.logpdf(x_is) - (-1.0 * (x_is - 4))
weights = np.exp(log_weights)
is_est = np.mean(weights)
```

(a) Explain why naive MC performs poorly for this problem. How many of the 100,000 standard normal samples are expected to exceed 4?

Naive Monte Carlo performs poorly for this problem because estimating $P(Z > 4)$ is an extreme rare-event simulation. When we draw samples from a standard normal distribution, the vast majority of the samples will fall between -3 and 3. Very few samples will actually land in the region of interest ($Z > 4$), meaning almost every sample contributes exactly 0 to the sum. This results in an estimator with massive relative variance. The expected number of samples exceeding 4 out of 100,000 is simply the number of trials multiplied by the true probability:

$$E[\text{count}] = n \times P(Z > 4)$$

$$E[\text{count}] \approx 100,000 \times (3.17 \times 10^{-5}) \approx 3.17$$

We can only expect to see about 3 valid samples out of 100,000. If randomness dictates you see 1, or 5, your estimate swings wildly, proving why naive MC is inefficient here.

(b) The importance sampling proposal is $q(x) = e^{-(x-4)}$ for $x \geq 4$. Verify that `log_weights` correctly computes $\log[\phi(x)/q(x)]$ where ϕ is the standard normal density.

In importance sampling, the weight for a sample x is the ratio of the target density $p(x)$ to the proposal density $q(x)$. Here, the target $p(x)$ is the standard normal density $\phi(x)$, and the proposal is $q(x) = e^{-(x-4)}$. The log of the weight is:

$$\log(w(x)) = \log\left(\frac{\phi(x)}{q(x)}\right) = \log(\phi(x)) - \log(q(x))$$

Looking at the proposal density $q(x) = e^{-(x-4)}$:

$$\log(q(x)) = \log(e^{-(x-4)}) = -(x-4)$$

Substituting this back into the log weight equation:

$$\log(w(x)) = \log(\phi(x)) - (-(x-4))$$

This matches the code exactly. By computing weights in log-space and exponentiating at the end, the code safely avoids numerical underflow issues that are common when multiplying very small probabilities.

(c) A classmate suggests using a t-distribution with 3 degrees of freedom as the proposal instead of the shifted exponential. Would this be a good choice for estimating $P(Z > 4)$? Why or why not?

No, using a standard t-distribution with 3 degrees of freedom would not be a good choice for this specific problem. A t_3 distribution is still centered at 0. If we draw 100,000 samples from it, approximately 50,000 will be negative, and the vast majority will still fall near the origin. We are still wasting an enormous amount of computational effort generating samples that do not satisfy $Z > 4$.

(d) Another classmate proposes using $N(5, 0.5^2)$ as the proposal. Compute the effective sample size (ESS) and compare it to using the shifted exponential. Which proposal is better and why?

The shifted exponential proposal is significantly better, providing over 6 times the effective statistical information per sample compared to the $N(5, 0.5^2)$ proposal.

The $N(5, 0.5^2)$ proposal has lighter tails than the standard normal target. As values get larger, the proposal's probability drops to zero much faster than the target's. This causes occasional importance weights to explode toward infinity, which completely dominates the average and crashes the Effective Sample Size (ESS).

The $Exp(1) + 4$ proposal has heavier tails than the standard normal target. Because it decays much slower, the importance weights strictly decrease as values get larger. The weights remain bounded and stable, yielding a high ESS.

```
# Template for part (d)
np.random.seed(42)
n_is = 10000

# Shifted exponential proposal (all samples are >= 4 by construction)
x_exp = np.random.exponential(1.0, n_is) + 4.0
log_w_exp = stats.norm.logpdf(x_exp) + (x_exp - 4)
log_w_exp -= np.max(log_w_exp)
w_exp = np.exp(log_w_exp)
w_exp_norm = w_exp / np.sum(w_exp)
ess_exp = 1.0 / np.sum(w_exp_norm ** 2)

# N(5, 0.5^2) proposal: multiply weights by indicator I(x > 4)
x_norm = np.random.normal(5, 0.5, n_is)
log_w_norm = stats.norm.logpdf(x_norm) - stats.norm.logpdf(x_norm, 5, 0.5)
w_norm_raw = np.exp(log_w_norm) * (x_norm > 4)
w_norm_raw[w_norm_raw > 0] /= np.max(w_norm_raw[w_norm_raw > 0])
w_norm_norm = w_norm_raw / np.sum(w_norm_raw)
ess_norm = 1.0 / np.sum(w_norm_norm ** 2)

print(f"{'ESS Shifted Exponential'}: {ess_exp}")
print(f"{'ESS Normal'}: {ess_norm}")
```

```
ESS Shifted Exponential: 4137.000460254567
ESS Normal: 677.4226510447006
```

Problem 5: The Curse of Dimensionality

Deterministic quadrature rules extend to multiple dimensions via product grids. For a d-dimensional integral with n nodes per dimension, the total number of function evaluations is n^d. Monte Carlo, by contrast, uses a fixed sample size regardless of dimension. In this problem, you will observe the crossover point where MC becomes more efficient than product-rule quadrature.

Consider the integral $I_d = \int_{[0,1]^d} e^{-\|\mathbf{x}\|^2} \, d\mathbf{x}$ where $\|\mathbf{x}\|^2 = x_1^2 + \cdots + x_d^2$.

(a) Write a function that computes I_d using a product trapezoidal rule with n nodes per dimension.

```
import numpy as np
import itertools
from scipy import integrate

def trap_nd(f, d, n):
    """Product trapezoidal rule on^d with n nodes per dimension.

    Parameters
    ----------
    f : callable
        Function from R^d to R. Takes an array of shape (N, d)
        where N is the number of evaluation points.
    d : int
        Number of dimensions.
    n : int
        Number of nodes per dimension.

    Returns
    -------
    float
        Approximate integral value.
    """
    x_1d = np.linspace(0.0, 1.0, n)
    h = 1.0 / (n - 1)

    w_1d = np.full(n, h)
    np.put(w_1d, 0, h / 2.0)
    np.put(w_1d, -1, h / 2.0)

    nodes = np.array(list(itertools.product(x_1d, repeat=d)))
    weight_combinations = np.array(list(itertools.product(w_1d, repeat=d)))

    w_nd = np.prod(weight_combinations, axis=1)

    y = f(nodes)
    return float(np.sum(y * w_nd))

# --- Testing and Error Reporting ---

ref_1d, _ = integrate.quad(lambda x: np.exp(-x**2), 0.0, 1.0)

def f(x):
    return np.exp(-np.sum(x**2, axis=1))
ds = (1, 2, 3, 4, 5, 6)
n_nodes = 10

print(f"{'d':>2} | {'Evals':>10} | {'Estimate':>15} | {'Abs Error':>15}")
print("-" * 50)

for d in ds:
    est = trap_nd(f, d, n_nodes)
    true_val = float(ref_1d ** d)

    err = float(abs(est - true_val))
    evals = n_nodes ** d

    print(f"{d:2d} | {evals:10d} | {est:15.8f} | {err:15.8e}")
```

```
 d |      Evals |        Estimate |       Abs Error
--------------------------------------------------
 1 |         10 |      0.74606687 |  7.57264900e-04
 2 |        100 |      0.55661577 |  1.13051395e-03
 3 |       1000 |      0.41527259 |  1.26580069e-03
 4 |      10000 |      0.30982112 |  1.25980185e-03
 5 |     100000 |      0.23114727 |  1.17546708e-03
 6 |    1000000 |      0.17245132 |  1.05290690e-03
```

The integrand separates as $e^{-\|\mathbf{x}\|^2} = \prod_{j=1}^{d} e^{-x_j^2}$, so the exact value is $I_d = \left(\int_0^1 e^{-x^2}\,dx\right)^d$. Use `scipy.integrate.quad` to compute the 1D reference value, then raise it to the d-th power. Compute the product trapezoidal estimate for $d = 1, 2, 3, 4, 5, 6$ with $n = 10$ nodes per dimension. For each d, report the number of function evaluations and the absolute error.

(b) For each dimension d above, estimate I_d using Monte Carlo with the same number of function evaluations as the product trapezoidal rule (i.e., n^d MC samples). Use `seed=42` with `numpy.random.default_rng`. Compare the MC error to the trapezoidal error for each

d. At what dimension does MC become more accurate?

```
def mc_nd(f, d, num_evals, seed=42):
    """Monte Carlo integration on^d."""
    rng = np.random.default_rng(seed)
    U = rng.uniform(0.0, 1.0, (num_evals, d))
    y = f(U)
    estimate = np.mean(y)
    return float(estimate)

ref_1d, _ = integrate.quad(lambda x: np.exp(-x**2), 0.0, 1.0)

def f(x):
    return np.exp(-np.sum(x**2, axis=1))

ds = (1, 2, 3, 4, 5, 6)
n_nodes = 10

print(f"{'d':>2} | {'Evals':>10} | {'Trap Error':>15} | {'MC Error':>15}")
print("-" * 55)

for d in ds:
    evals = int(n_nodes ** d)
    true_val = float(ref_1d ** d)

    # Trapezoidal Estimate
    est_trap = trap_nd(f, d, n_nodes)
    err_trap = float(abs(est_trap - true_val))

    # Monte Carlo Estimate (matching the exact computational budget)
    est_mc = mc_nd(f, d, evals, seed=42)
    err_mc = float(abs(est_mc - true_val))

    print(f"{d:2d} | {evals:10d} | {err_trap:15.8e} | {err_mc:15.8e}")
```

```
 d |      Evals |      Trap Error |        MC Error
-------------------------------------------------------
 1 |         10 |  7.57264900e-04 |  7.24072577e-02
 2 |        100 |  1.13051395e-03 |  1.00652952e-02
 3 |       1000 |  1.26580069e-03 |  8.35092459e-04
 4 |      10000 |  1.25980185e-03 |  1.26047788e-03
 5 |     100000 |  1.17546708e-03 |  6.15935834e-04
 6 |    1000000 |  1.05290690e-03 |  1.01604814e-06
```

MC becomes more accurate at d = 3 and above.

Close

```
**(c)** The separability $e^{-\|\mathbf{x}\|^2} = \prod_j e^{-x_j^2}$ means one could compute the $d$-dimensional integral as a pro
one-dimensional integrals, each requiring only $n$ nodes, for
$n \cdot d$ evaluations instead of $n^d$. Explain why this tr
generalize: give an example of a non-separable integrand on $
cannot be decomposed this way, and explain why such integrand
in statistical applications involving correlated random effe

1. Why the Separability Trick Fails to GeneralizeThe computat
shortcut of performing $d$ one-dimensional integrals (requir
d$ evaluations) relies entirely on the assumption of structur
independence. The integrand must factor perfectly into univar
functions: $f(\mathbf{x}) = \prod_{j=1}^d f_j(x_j)$. If the v
interact or are coupled mathematically, this factorization br
and the integral cannot be decomposed.

2. Example of a Non-Separable IntegrandConsider an integrand
multiplicative interaction term on $^2$:$$f(x_1, x_2) = \exp
$$There are no two strictly univariate functions $g(x_1)$ and
such that $g(x_1)h(x_2) = \exp(x_1 x_2)$ for all $x_1, x_2 \i
the dimensions cannot be isolated, you are forced to evaluate
joint space, requiring the full $n^2$ grid.

3. Relevance to Statistical ApplicationsNon-separable integra
ubiquitous in modern statistics, particularly in hierarchical
mixed-effects models, due to correlated random effects. When
repeated measures or longitudinal data, subject-specific rand
$\mathbf{b} = (b_1, \dots, b_d)^T$ are typically assumed to
```

(c) The separability $e^{-\|\mathbf{x}\|^2} = \prod_j e^{-x_j^2}$ means one could compute the d-dimensional integral as a product of d one-dimensional integrals, each requiring only n nodes, for a total of $n \cdot d$ evaluations instead of n^d. Explain why this trick does not generalize: give an example of a non-separable integrand on $[0, 1]^d$ that cannot be decomposed this way, and explain why such integrands are common in statistical applications involving correlated random effects.

1. Why the Separability Trick Fails to GeneralizeThe computational shortcut of performing d one-dimensional integrals (requiring $n \cdot d$ evaluations) relies entirely on the assumption of structural independence. The integrand must factor perfectly into univariate functions: $f(\mathbf{x}) = \prod_{j=1}^{d} f_j(x_j)$. If the variables interact or are coupled mathematically, this factorization breaks down, and the integral cannot be decomposed.

2. Example of a Non-Separable IntegrandConsider an integrand with a simple multiplicative interaction term on ^{2}:
$$f(x_1, x_2) = \exp(x_1 x_2)$$
There are no two strictly univariate functions $g(x_1)$ and $h(x_2)$

multivariate normal distribution: $\mathbf{b} \sim N(\mathbf{0}, \boldsymbol{\Sigma})$. If the random effects are correlated (e.g., a patient's baseline intercept correlates with their slope over time), the covariance matrix $\boldsymbol{\Sigma}$ contains non-zero off-diagonal elements. The probability density function includes the quadratic form $\mathbf{b}^T \boldsymbol{\Sigma}^{-1} \mathbf{b}$, which generates cross-product interaction terms (e.g., $c_{12} b_1 b_2$). These cross-terms irreversibly entangle the variables within the exponent, making the marginal likelihood integral completely non-separable. This mathematical entanglement is why deterministic product rules fail in high-dimensional statistics, cementing Monte Carlo methods (like MCMC) as the standard solution.

such that $g(x_1)h(x_2) = \exp(x_1 x_2)$ for all $x_1, x_2 \in$. Because the dimensions cannot be isolated, you are forced to evaluate the full joint space, requiring the full n^2 grid.

3. Relevance to Statistical ApplicationsNon-separable integrands are ubiquitous in modern statistics, particularly in hierarchical or mixed-effects models, due to correlated random effects. When modeling repeated measures or longitudinal data, subject-specific random effects $\mathbf{b} = (b_1, \dots, b_d)^T$ are typically assumed to follow a multivariate normal distribution: $\mathbf{b} \sim N(\mathbf{0}, \boldsymbol{\Sigma})$. If the random effects are correlated (e.g., a patient's baseline intercept correlates with their slope over time), the covariance matrix $\boldsymbol{\Sigma}$ contains non-zero off-diagonal elements. The probability density function includes the quadratic form $\mathbf{b}^T \boldsymbol{\Sigma}^{-1} \mathbf{b}$, which generates cross-product interaction terms (e.g., $c_{12} b_1 b_2$). These cross-terms irreversibly entangle the variables within the exponent, making the marginal likelihood integral completely non-separable. This mathematical entanglement is why deterministic product rules fail in high-dimensional statistics, cementing Monte Carlo methods (like MCMC) as the standard solution.